

SOFTWARE REQUIREMENT SPECIFICATION

(Specialization Project - C1 Evaluation)

Sentari Agent-OS

*(An Operating-System-Inspired Kernel for Scheduling, Isolation,
and Deadlock Detection in Multi-Agent AI Systems)*

GitHub Repository: <https://github.com/Vishalsym/Sentari-AgentOS>

Submitted by

Vishal Sharma

Roll No: 25225039

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

School of Sciences

2026-2027

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

1 Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for Sentari Agent-OS, a kernel layer that manages AI agents as scheduled, resource-bounded, isolated processes. It is intended for the project mentor/evaluator and for the developer (the author) as the authoritative reference during design and implementation.

1.2 Scope

Sentari Agent-OS operates as a middleware layer sitting between an agent-orchestration framework (e.g., LangGraph, CrewAI) and the external resources those agents consume (LLM APIs, tools, a shared knowledge base). It is responsible for admission, scheduling, syscall mediation, memory isolation, deadlock detection, and preemption of agents. It is not responsible for the internal reasoning quality of any individual agent — that remains the responsibility of the wrapped framework/model.

1.3 Definitions and Abbreviations

Term	Definition
PCB	Process Control Block — the kernel's record of an agent's state, priority, and quota
Syscall	A mediated request from an agent to the kernel for a tool call, memory access, or spawn/yield
Wait-for graph	A directed graph where an edge $A \rightarrow B$ means agent A is blocked waiting on agent B
Quota	The remaining token / API-call budget assigned to an agent for its lifetime or time-slice
Preemption	Kernel-forced suspension of a running agent before it voluntarily yields

1.4 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (structure reference).
- Silberschatz, Galvin, Gagne — Operating System Concepts (scheduling, deadlock, virtual memory chapters).
- Author's own Yalnix kernel coursework (process scheduling and PCB implementation).

2 Overall Description

2.1 Product Perspective

Sentari Agent-OS is a new, self-contained middleware component. It exposes a syscall API that agent frameworks call into, and internally manages PCBs, a scheduler queue, a wait-for graph, and a syscall audit log. It is designed to be framework-agnostic: any orchestration layer capable of routing its tool calls through a Python interface can be wrapped.

2.2 Product Functions (Summary)

- Register and admit agents into the kernel with an initial priority and quota.
- Schedule agents fairly for access to the LLM API / tool-execution resource.
- Mediate every tool/API/memory access through a syscall interface.
- Detect and resolve deadlocks between mutually waiting agents.
- Preempt and terminate agents that violate quota, timeout, or deadlock conditions.
- Provide isolated per-agent context with controlled access to shared knowledge.
- Log all syscalls and scheduling decisions for audit and evaluation.

2.3 User Characteristics

The primary user of Sentari Agent-OS is a developer integrating it into an existing or new multi-agent pipeline. No end-user-facing UI is required for core functionality; an optional dashboard serves observability needs during development and demonstration.

2.4 Constraints

- Must be implementable and demonstrable within a single-semester specialization project timeline.
- Must work against at least one real LLM provider API with real rate limits.
- Kernel overhead must remain small relative to LLM call latency (target: <5% added latency per syscall).

2.5 Assumptions and Dependencies

- Agents are cooperative in the sense that they issue syscalls rather than bypassing the kernel (a wrapped framework enforces this at the integration boundary).
- A stable internet connection and valid API credentials are available for the LLM provider used in evaluation.

3 Functional Requirements

3.1 Agent Admission & PCB Management

ID	Requirement
FR-1	The system shall create a PCB for each new agent containing: agent ID, state, priority, token quota, owned resources, and parent/child links.
FR-2	The system shall support agent states: NEW, READY, RUNNING, BLOCKED, TERMINATED, KILLED.
FR-3	The system shall allow an agent to spawn child agents via a syscall, with the child inheriting a bounded share of the parent's quota.

3.2 Scheduling

ID	Requirement
FR-4	The system shall schedule READY agents for API-call turns using a configurable policy (round-robin or priority-based).
FR-5	The system shall enforce a maximum time-slice / call-count per turn before returning a running agent to READY.
FR-6	The system shall support dynamic priority adjustment (e.g., aging) to prevent starvation of low-priority agents.

3.3 Syscall Mediation

ID	Requirement
FR-7	The system shall expose a syscall interface (tool_call, memory_read, memory_write, spawn, yield) as the sole path to external resources.
FR-8	The system shall validate every syscall against the issuing agent's permissions and remaining quota before execution.
FR-9	The system shall log every syscall (agent, type, arguments, timestamp, result) to a persistent audit log.

3.4 Memory Isolation

ID	Requirement
FR-10	The system shall give each agent an isolated context store not directly writable by any other agent.
FR-11	The system shall provide a shared knowledge-base region accessible only through explicit memory_read/memory_write syscalls.

3.5 Deadlock Detection & Recovery

ID	Requirement
FR-12	The system shall maintain a wait-for graph whenever an agent blocks pending another agent's output.
FR-13	The system shall run cycle detection over the wait-for graph on every new blocking edge.
FR-14	On detecting a cycle, the system shall select a victim agent (e.g., lowest priority) and terminate or roll it back to break the cycle.

3.6 Preemption & Termination

ID	Requirement
FR-15	The system shall preempt an agent that exceeds its allotted time-slice or call quota.
FR-16	The system shall forcibly kill an agent that remains blocked beyond a configurable timeout or is a deadlock victim.
FR-17	The system shall release all resources held by a terminated/killed agent.

4 Non-Functional Requirements

Category	Requirement
Performance	Syscall mediation shall add no more than ~5% latency overhead relative to a direct (unmediated) tool call.
Reliability	The kernel shall not crash or hang due to a single agent's failure; failures shall be isolated to that agent.
Scalability	The scheduler shall support at least 20 concurrently registered agents in the reference implementation.
Security	Agents shall have no direct access to raw API keys or unmediated network calls; all access is via syscalls.
Auditability	Every scheduling decision and syscall shall be reconstructable from the persisted log.
Maintainability	Each kernel subsystem (scheduler, syscall layer, memory manager, deadlock detector) shall be independently testable.
Usability	The optional dashboard shall present agent state, quota usage, and the wait-for graph without requiring log-file inspection.

5 External Interface Requirements

5.1 Software Interfaces

- LLM Provider API (e.g., Anthropic Messages API) — accessed only through the syscall layer's tool_call mediation.
- Wrapped agent framework (LangGraph / CrewAI) — integrates by routing its tool-execution step through the Sentari Agent-OS syscall client.
- SQLite — persistence for PCB snapshots and the syscall audit log.

5.2 Hardware Interfaces

None beyond a standard development machine with internet access; no specialized hardware is required.

5.3 User Interfaces

A minimal web dashboard (optional, Section design document) displaying active agents, their PCB state, quota usage, the current wait-for graph, and a live syscall log stream.

6 Use Cases (Summary)

Use Case	Primary Actor	Description
Admit Agent	Orchestration framework	Registers a new agent and creates its PCB with initial priority/quota.
Request Tool Call	Agent	Agent issues a syscall to invoke an external tool; kernel validates and mediates.
Detect Deadlock	Kernel (Deadlock Detector)	Kernel identifies a wait-for cycle and selects/terminates a victim agent.
Preempt Runaway Agent	Kernel (Scheduler)	Kernel forcibly suspends/kills an agent exceeding its quota or timeout.
View System State	Developer	Developer inspects live PCB table, wait-for graph, and syscall log via dashboard.